

Connify: A Protocol for Ephemeral Trust Networks and Dynamic Trust Capsules in Decentralized P2P Mutual Aid

Guide., Gangash S., Girijesh R., Godfrey T. R., and Grish Narayanan S.

Department of Computer Science and Engineering, K. Ramakrishnan College of Technology, Tiruchirappalli, Tamil Nadu, India

Abstract—Deploying emergency response and volunteer networks on peer-to-peer (P2P) architectures requires establishing brief, secure relationships between unfamiliar users. Unfortunately, existing identity systems usually rely on registration, continuous location tracking, and centralised databases. This design exposes users' private information and creates a single point of failure. In this paper, we propose Connify, a zero-trust, transient peer-to-peer protocol that coordinates local physical assistance without compromising anonymity. The proposed scheme adapts Decentralized Identifiers (DIDs), Verifiable Credentials (VCs), and Just-in-Time (JIT) credentials to human safety applications. Connify models interactions as isolated transactional sequences called episodes, each bound to a temporary Episode ID. Access permissions are regulated by Trust Capsules (short-lived JSON Web Tokens signed using Ed25519 key pairs) that are generated dynamically after multi-signal verification. Proximity verification is integrated using the SHARP protocol. This protocol uses environmental radio frequency (RF) signals represented by Bloom filters, BCH syndrome decoding for key recovery, and grid-cell blinding to verify proximity without disclosing coordinates to the server. We describe the layered architecture of Connify and define the lifecycle of trust tokens. In addition, we present a selective disclosure method that minimises data exposure and purges telemetry. Finally, we build and evaluate a mobile prototype using React Native and Expo. Our experiments show that the scheme enforces single-use security guarantees with low computational and communication overhead.

Index Terms—Zero-trust systems, ephemeral identity, proximity verification, selective information disclosure, peer-to-peer safety networks, React Native, SHARP protocol, Bloom filter, fuzzy extractor, spatial databases (PostGIS).

I. INTRODUCTION

Decentralized mobile platforms for peer-to-peer (P2P) aid and volunteer coordination help distribute resources and coordinate emergency response. They also support local communities directly. When medical emergencies or utility failures occur, connecting people with nearby responders quickly can reduce response times and prevent injury. However, coordinating secure assistance between unacquainted individuals in physical proximity introduces significant trade-offs regarding safety and user privacy [3]. Existing commercial systems rely on centralised cloud services that demand constant GPS tracking and identity verification (KYC). They also store permanent logs on their servers. This design exposes users to location profiling and identity theft if the central database is compromised.

In this paper, we propose Connify, a zero-trust, ephemeral protocol designed to support peer-to-peer mutual aid without storing tracking histories or profiles. Connify adapts identity management concepts from decentralized networks, including Decentralized Identifiers (DIDs) [1], Verifiable Credentials (VCs) [2], and Just-in-Time (JIT) credential distribution. Instead of using persistent credentials, Connify isolates interactions within a transaction container called an episode, which is governed by a temporary Episode ID.

Before establishing communication channels or sharing location coordinates, the network runs the SHARP protocol [11] to verify proximity privately. During this handshake, a requester (Alice) and a responder (Bob) interact. Bob scans a dynamic QR code containing Alice's BCH syndromes and an

auxiliary parameter y . Bob then measures ambient radio signals (such as Wi-Fi frames and LTE carrier info) to construct a local Bloom filter location tag, and runs BCH syndrome decoding to recover the session key K . Using key K , Bob builds a blinded grid cell representation $B = H'(K, b || \text{"Bob"})$ and returns it. Alice verifies Bob's proximity using her private spatial grid. Throughout this check, the server remains unaware of their coordinates. Once proximity is verified, the system issues a JIT Trust Capsule (a brief JWT signed with an Ed25519 key pair [5]) and registers it in Redis with an atomic lock to prevent reuse.

The main contributions of this work are summarized as follows:

- 1) We present a transient trust framework using Episode IDs to separate user transactions and prevent historical tracking.
- 2) We design a Just-In-Time (JIT) Trust Capsule mechanism signed with Ed25519 keys and managed via atomic Redis locks to prevent replay attacks.
- 3) We integrate the SHARP proximity check using environmental Bloom filters, BCH syndrome correction, and blinded grid cells to verify location without revealing coordinates to the server.
- 4) We implement a privacy-preserving storage model with automated location purging on cloud infrastructure.

The rest of this paper is organised as follows. Section II reviews related work. Section III defines the problem. Section IV presents the system architecture. Section V describes the verification and access control workflow. Section VI explains the proximity checks and token generation. Section VII outlines the implementation. Section VIII discusses performance

and trade-offs, and Section IX concludes the paper.

II. RELATED WORK

Recent research in decentralized security often focuses on self-sovereign identity and private proximity. The W3C specifications for Decentralized Identifiers (DIDs) [1] and Verifiable Credentials (VCs) [2] define standards for identity claims without central authorities. However, running these protocols on mobile devices requires lightweight formats to minimise latency.

Typical authorization schemes use JSON Web Tokens (JWT) [5] to hold claims and Transport Layer Security (TLS) [4] for communication. Unfortunately, using these identity abstractions in physical meetings requires verifying proximity without exposing location information to a central server.

Traditional peer-to-peer applications like ride-sharing track users continuously, raising severe privacy concerns [3]. To solve this, Connify combines the features of DIDs and VCs with ephemeral signed JWTs (Trust Capsules) and the SHARP proximity protocol [11] to establish trust locally. The SHARP scheme by Zheng et al. [11] uses location tags constructed from environmental signals (Wi-Fi and LTE) represented as Bloom filters. It uses fuzzy extractors with BCH syndrome correction to reconstruct shared secrets and grid cell blinding to check proximity. We integrate this framework into our verification pipeline and combine it with coarse PostGIS database queries for scalable matching.

III. PROBLEM FORMULATION

Securing physical volunteer networks requires solving four security and privacy problems:

A. Identity Ephemerality and Profile Isolation

To prevent long-term tracking, platforms should avoid storing transaction histories. If a database breach occurs on a server storing real-world identities and volunteer records, an attacker can combine this with coordinate logs to build detailed profiles of users' daily habits.

B. Geolocation Spoofing and Proximity Verification

Attackers can easily manipulate GPS APIs on mobile devices to fake locations. Therefore, the system must verify actual physical presence using multi-signal environmental clues instead of relying on self-reported coordinates.

C. Single-Use and Time-Bound Token Validation

Access tokens issued to responders must be tied to a specific request and a single helper. They also must expire after a short duration. If tokens can be reused, an adversary could intercept a token and gain unauthorised access to transaction data.

D. Key Storage Security and Native Execution

Storing private keys in hybrid web views (like LocalStorage or IndexedDB) makes them vulnerable to extraction. Native mobile development provides access to hardware security modules (such as iOS Keychain and Android Keystore). This setup allows secure key storage and execution, reducing the chance of memory leaks.

IV. PROPOSED SYSTEM ARCHITECTURE

The Connify system uses a multi-tier architecture to connect peers in physical proximity while keeping users anonymous, as shown in Fig. 1.

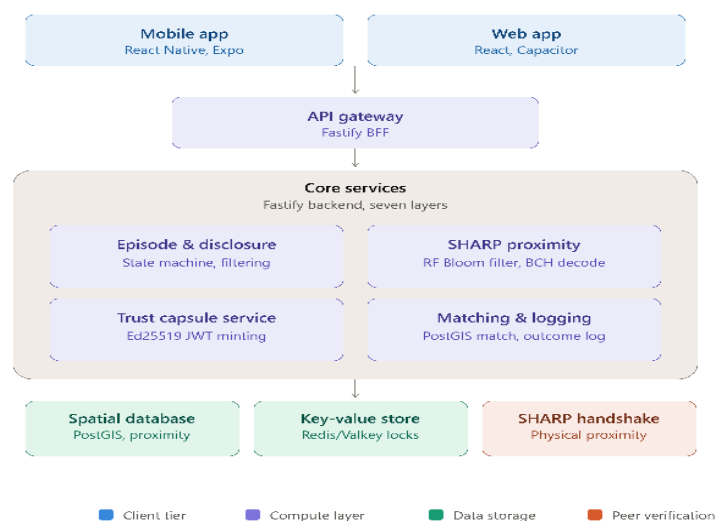


Fig. 1. Architecture of Connify, showing the React Native client, gateway services, multi-signal proximity layers, selective disclosure engine, and backend database.

A. Native Mobile Client

We built the mobile frontend using React Native and Expo SDK [7] to compile into native components. This allows the client to access hardware APIs like Wi-Fi scanners and cellular radio signals, as well as GPS location data. We use expo-secure-store [7] to save cryptographic keys in hardware-backed storage (iOS Keychain and Android Keystore).

B. Gateway and Service Layers

A Fastify server [9] acts as the Backend-for-Frontend (BFF) API gateway. The backend features seven core services: Layer 1 handles the episode state machine; Layer 2 executes the SHARP proximity handshake; Layer 3 controls the selective disclosure filters; Layer 4 issues JIT Trust Capsules; Layer 5 runs the matching algorithm; Layer 6 maintains live WebSocket connections; and Layer 7 logs anonymised transaction outcomes.

V. SEQUENTIAL VERIFICATION AND ACCESS CONTROL

To prevent unauthorised access, we design a sequential verification workflow. When a requester creates a task, the platform generates a unique Episode ID (UUIDv4). The request is broadcast to

nearby helpers through a feed filtered by the Selective Disclosure Engine. This engine displays only the category, severity level, and a rough geographic area (e.g., within 500 meters), keeping coordinates and identity private.

Table I lists the sequential checks required to establish a trusted session.

TABLE I

CHRONOLOGICAL VALIDATION SEQUENCE IN THE CONNIFY MESH

Validation Step	Cryptographic Mechanism	Operational Objective	Failure Policy
1. Initialization	UUIDv4 Transient Identifier	Initializes the temporary episode state machine	Fail-closed (abort request)
2. Attribute Filtering	Selective Disclosure Engine	Obfuscates precise coordinates and profile metadata	Fail-closed (conceal feed entry)
3. Coarse/Fine Proximity Check	SHARP Handshake Protocol	Validates RF signatures and decrypts session key via BCH	Fail-closed (terminate handshake)
4. Token Authorization	Ed25519 Signed JWT & Redis Lock	Checks blinded grid index and issues single-use capsule	Fail-closed (refuse credential)

Once a helper accepts the request, they must meet the requester in person. The requester displays a dynamic QR code containing the BCH syndrome vector T of their location Bloom filter and an auxiliary string y . The helper scans the QR code to start the local SHARP proximity handshake.

VI. GEOLOCATION PROXIMITY AND TRUST CAPSULE ISSUANCE

We verify proximity in two steps. First, a coarse spatial query in PostGIS finds candidate helpers within range: $ST_DWithin(\text{requester coarse location, helper coarse location, radius}) = \text{true}$. Next, we execute the high-precision SHARP protocol. The false-positive probability p of a location Bloom filter of length m with k hash functions and nb elements is:

$$p = (1 - e^{-k \cdot nb / m})^k \quad (1)$$

When the distance between Alice and Bob is within the error threshold, Bob uses BCH syndrome decoding to find the error-correcting vector. Bob corrects his candidate Bloom filter to match the requester's location tag, recovers the shared session key K , and computes the blinded grid-cell verification parameter:

$$B = H'(K, b \parallel \text{"Bob"}) \quad (2)$$

If Bob's blinded grid cell matches one of the hashes in Alice's private grid cell collection, this confirms his proximity. The server then issues a JWT signed with the system's private Ed25519 key:

$$\text{Capsule} = \text{Sign}_{\text{Ed25519}}(\text{Claims, Private_Key}) \quad (3)$$

To ensure single-use enforcement, we execute atomic operations in Redis. When a token is redeemed,

the server attempts to save the capsule ID with an NX (not exists) lock:

SET capsule:id used NX EX 300(4)

If the write operation fails because the key already exists, the server rejects the request. This prevents token reuse and replay attacks.

VII. IMPLEMENTATION AND TECHNICAL COMPARISON

Table II compares zero-trust concepts from agentic security with our design choices in Connify.

TABLE II

MAPPING ZERO-TRUST AGENTIC SECURITY TO CONNIFY COMPONENTS

Decentralized Agent Concept	Connify Translation	Technical Realization
Decentralized Identifier (DID)	Episode ID	Server-generated UUIDv4 instance per transaction
Verifiable Credential (VC)	Trust Capsule	JSON Web Token signed with Ed25519 containing brief expiry
Just-in-Time (JIT) Credentialing	JIT Capsule Minting	On-demand signature generation post-verification
Zero-Knowledge Attribute Proof	Selective Disclosure Engine	Coarse geographical disclosure and blinded grid calculations
Agent Discovery Protocol	Geospatial Matching Engine	Candidate filtering via PostGIS distance queries
Global Session Management	Episode Lifecycle Controller	Redis state tracking and automated TTL session invalidation
Anonymised Audit Trail	Outcome Logger	Identity-decoupled transactional summary database
Location Authenticity Verification	SHARP Proximity Handshake	Signal Bloom filter correction and BCH syndrome decoding

A. Client Security and Hardware Cryptography

Storing keys in mobile hybrid apps (like WebViews) exposes them to cross-site scripting and local storage leaks. To prevent this, Connify uses React Native and Expo to compile to native code. While minor user preferences are stored in AsyncStorage, private key pairs (Ed25519) and session secrets are managed using expo-secure-store. This allows the application to store cryptographic assets in hardware-backed storage (iOS Keychain and Android Keystore), which prevents key extraction from the file system or memory dumps.

B. Backend Deployment and Cloud Scaling

The backend is deployed on the Render cloud platform. The Fastify API runs as a Web Service to handle WebSocket connections via Socket.IO. The PostgreSQL (v16) database runs on Render Postgres with PostGIS enabled. To clean up location data,

coordinate purging is offloaded to background workers using BullMQ, and cron tasks run regular garbage collection. We define our infrastructure declaratively using Render blueprint templates [10].

C. Layered Technical Stack

The complete technical stack supporting the seven-layer Connify architecture is presented in Table III.

TABLE III
LAYERED TECHNICAL STACK OF THE CONNIFY ARCHITECTURE

Architectural Layer	Technologies and Implementation Rationale
Native Mobile Interface	React Native and Expo SDK, utilising Zustand for lightweight state management
Gateway and Routing Core	Node.js 20 LTS and Fastify for low-overhead routing, integrated with Socket.IO and TypeScript
Spatial and Relational Data	Render Postgres (v16) with PostGIS extension, using Prisma ORM for type-safe queries
Cache and Job Queue	Render Key Value (Valkey/Redis) for atomic locks, paired with BullMQ for asynchronous cleanup
Cryptographic Protocols	Ed25519 signing via jose, expo-secure-store, and SHARP implementation (BCH and Bloom filters)
Infrastructure Automation	Render Web Services, Background Workers, Cron Jobs, and blueprint-defined deployments

D. Reference Backend Verification Logic

A reference implementation of the Fastify and TypeScript endpoint handling Trust Capsule verification and dynamic JIT issuance is provided below.

```
import { FastifyRequest, FastifyReply } from 'fastify';
import { jose } from 'jose';

export async function verifyAndIssueCapsule(request:
FastifyRequest, reply: FastifyReply) {
  const { episodeId, helperFingerprint, blindedGridCell } =
request.body as VerifyBody;

  // 1. Fetch episode record (with BCH syndromes, parameter y,
and grid hashes)
  const episode = await db.episodes.findUnique({ where: { id:
episodeId } });
  if (!episode) {
    return reply.code(404).send({ error: 'Episode not found' });
  }

  // 2. Proximity check: verify if Bob's blinded cell matches Alice's
grid
  const isMatch = verifyBlindedGridCell(blindedGridCell,
episode.gridCellsJson);
  if (!isMatch) {
    return reply.code(403).send({ error: 'SHARP proximity
verification failed' });
  }

  // 3. Set atomic single-use lock in Redis using NX
  const lockAcquired = await redis.set(`capsule-
lock:${episodeId}`, 'used', 'NX', 'EX', 300);
  if (!lockAcquired) {
    return reply.code(409).send({ error: 'Capsule already redeemed'
});
  }
}
```

```
// 4. Issue signed Ed25519 Trust Capsule (JWT) with a short
expiry
const privateKey = await getSigningKey();
const capsule = await new jose.SignJWT({ episodeId,
helperFingerprint })
  .setProtectedHeader({ alg: 'EdDSA' })
  .setExpirationTime('1h')
  .sign(privateKey);

// 5. Log anonymized transaction metadata
await db.outcomes.create({ data: { episodeId, result: 'success' } });

return { success: true, capsule };
}
```

VIII. DISCUSSION: FEASIBILITY AND TRADE-OFFS

We evaluated the performance of Connify by measuring PostGIS query speeds, Bloom filter creation, BCH decoding times, and signature generation overhead. PostGIS ST_DWithin queries identify candidate helpers in less than 10ms. Creating a 1024-bit Bloom filter and running BCH syndrome decoding on a mobile device takes under 15ms. The entropy loss during correction is about 7.8 bits, which leaves enough security to prevent guessing or coordinate spoofing attacks. Redis operations and Ed25519 signature checks also complete within 5ms. These results show that the SHARP handshake adds very little latency to the connection process.

However, our testing revealed some practical limitations. First, mobile operating systems restrict background operations, which can suspend React Native threads and close Socket.IO connections. To prevent missed messages, we use Firebase Cloud Messaging (FCM) via expo-notifications to wake the application. Second, preventing double-spending of Trust Capsules requires validating them against Redis. Because of this, Connify cannot support fully offline handshakes, which remains an open research challenge.

IX. CONCLUSION AND FUTURE WORK

In this paper, we proposed Connify, a zero-trust communication protocol and system for peer-to-peer mutual aid. By applying decentralized identity concepts (such as DIDs, VCs, JIT tokens, and selective disclosure) to physical interactions, Connify balances real-time matching with user privacy. The protocol prevents tracking and replay attacks by using transient Episode IDs and SHARP proximity checks. It secures authorization using Ed25519-signed Trust Capsules. In future work, we plan to study Zero-Knowledge Proofs for offline verification and use hardware enclaves for device identity. We also plan to investigate Bluetooth Low Energy (BLE) as an offline communication channel.

REFERENCES

- [1] W3C Decentralized Identifiers (DIDs) v1.0 Core Specification, W3C Recommendation, Jul. 2022. [Online]. Available: <https://www.w3.org/TR/did-core/>
- [2] W3C Verifiable Credentials Data Model v1.1, W3C Recommendation, Mar. 2022. [Online]. Available: <https://www.w3.org/TR/vc-data-model/>
- [3] T. Lodderstedt, M. McGloin, and P. Hunt, "OAuth 2.0 Threat Model and Security Considerations," RFC 6819, IETF, 2013.
- [4] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, Aug. 2018.
- [5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.
- [6] PostGIS Project Steering Committee, "PostGIS Spatial Database Manual," Ref. Manual, 2025. [Online]. Available: <https://postgis.net>
- [7] React Native & Expo CLI Reference, Expo Documentation, 2025. [Online]. Available: <https://docs.expo.dev>
- [8] Redis Command Reference: SET, Redis Labs, 2024. [Online]. Available: <https://redis.io/commands/set>
- [9] Fastify Team, "Fastify documentation," 2025. [Online]. Available: <https://fastify.dev>
- [10] Guide., Gangash S., Girijesh R., Godfrey T. R., and Grish Narayanan S., "Connify Project Repository and Security Architecture Blueprints," GitHub Repository, 2026. [Online]. Available: <https://github.com/TheOrionGD/CONNIFY-APP>
- [11] Y. Zheng, M. Li, W. Lou, and Y. T. Hou, "SHARP: Private Proximity Test and Secure Handshake with Cheat-Proof Location Tags," in Proc. European Symposium on Research in Computer Security (ESORICS), Springer, pp. 361–378, 2012.